卓越工程综合训练

Comprehensive Practice for Outstanding Engineer Education Plan

课程编号：100076108



闫波

北京理工大学



设计模式

设计模式的概念与组成

- 模式化的过程就是把问题抽象化，在忽略掉不重要的细节后，发现问题的一般性本质，并找到普遍适用的解决方案的过程。
- 原则

  开-闭原则；里氏代换原则；依赖倒转原则；接口隔离原则；组合/聚合复用原则；迪米特法则；

设计模式的概念与组成

●软件设计模式是软件高效和成熟的程序设计模板，模板包含故有问题的解决方案和逻辑，强调复用程序结构。

●设计模式是对软件开发经验总结。一个设计模式有系统的命名、解释和评价，是某个重要的可重现的设计方案。

设计模式的概念与组成

- 设计模式的基本成分：

1）模式名称：用来描述一个设计问题，能表达和交流设计思想；

2）问题：能反映使用模式的必要性，必要条件，希望的特性；

3）解决方案：描述设计要素，要素的组织结构，相互关系；

4）后果：描述使用的优势和代价，与相关模式的区别；

设计模式的原则--开-闭原则

●一个软件实体应该对**扩展开放**，对**修改关闭**。软件系统中包含的各种组件，例如模块（Modules）、类（Classes）以及功能（Functions）等等，应该在不修改现有代码的基础上，引入新功能。

●实现开闭原则的关键就在于“**抽象**”。把系统的所有可能的行为抽象成一个抽象底层，这个抽象底层规定出所有的具体实现必须提供的方法的特征。作为系统设计的抽象层，要预见所有可能的扩展，从而使得在任何扩展情况下，系统的抽象底层不需修改；同时，由于可以从抽象底层导出一个或多个新的具体实现，可以改变系统的行为，因此系统设计对扩展是开放的。（Java抽象类 接口）

设计模式的原则--开-闭原则

●对于扩展是开放的（Open for extension）。这意味着模块的行为是可以扩展的。当应用的需求改变时，我们可以对模块进行扩展，使其具有满足那些改变的新行为。也就是说，我们可以改变模块的功能。

●对于修改是关闭的（Closed for modification）。对模块行为进行扩展时，不必改动模块的源代码或者二进制代码。模块的二进制可执行版本，无论是可链接的库、DLL或者.EXE文件，都无需改动。

设计模式的原则—里氏代换原则(Liskov Substitution Principle, LSP)



●里氏代换原则，任何基类可以出现的地方，子类一定可以出现。里氏代换原则是对“开-闭”原则的补充。如果S是T的子类型，那么在不改变程序正确性的前提下，T类型的对象可以被S类型的对象替换。简单来说，就是子类必须能够完全替代其父类，而不影响程序的正确性。

●问题由来：有一功能P1，由类A完成。现需要将功能P1进行扩展，扩展后的功能为P，其中P由原有功能P1与新功能P2组成。新功能P由类A的子类B来完成，则子类B在完成新功能P2的同时，有可能会导致原有功能P1发生故障。解决方案：当使用继承时，遵循里氏替换原则。类B继承类A时，除添加新的方法完成新增功能P2外，尽量不要重写父类A的方法，也尽量不要重载父类A的方法。

设计模式的原则—里氏代换原则(Liskov Substitution Principle, LSP)

- 核心思想：
  - 行为一致性：子类不应该改变父类的预期行为
  - 契约继承：子类必须遵守父类定义的行为契约
  - 可替换性：使用父类的地方可以透明地使用子类
- 优势：提高代码的可复用性；增强系统的可扩展性；减少因继承关系引入的错误；提高代码的可维护性；
- 实践：
  - 优先使用组合而非继承
  - 设计父类时要考虑所有可能的子类行为
  - 使用设计模式如策略模式、模板方法模式等来替代不合适的继承
  - 考虑使用接口来定义行为契约

里氏代换原则强调了良好的继承关系设计，是保证面向对象系统健壮性的重要原则之一。

设计模式的原则—依赖倒转原则(DIP)

●要依赖于抽象，不要依赖与具体。要针对接口编程，不要针对实现编程。高层模块不应该依赖于低层模块，二者都应该依赖于抽象。

●传统设计中，上层调用下层，上层依赖于下层，导致代码耦合度高，难以修改或替换低层模块，难以进行单元测试，系统扩展性差。

●实现：

- ●1. 通过接口/抽象类实现倒置
- ●2. 依赖注入。构造函数注入；属性/Setter注入；接口注入
- ●3. 使用依赖注入容器。现代框架如Spring、Guice等提供了依赖注入容器管理对象的创建和依赖关系。

设计模式的原则—依赖倒转原则(DIP)

- 优势：
  - 降低耦合度：模块间通过抽象交互，减少直接依赖
  - 提高可扩展性：易于添加新功能而不影响现有代码
  - 增强可测试性：便于使用mock对象进行单元测试
  - 提高代码复用性：抽象可以在不同上下文中重用
- 实践：
  - 不要过度设计，简单场景可以直接使用具体类
  - 合理设计抽象接口，避免接口过于庞大或频繁变更
  - 依赖注入框架要理解其原理，避免滥用
  - 在系统演进过程中逐步应用DIP

依赖倒转原则是构建灵活、可维护软件架构的关键，合理应用可以显著提高代码质量。

设计模式的原则—接口隔离原则(ISP)

●使用多个专用的接口比使用单一的总接口要好。

●客户端不应该依赖它不需要的接口；一个类对另一个类的依赖应该建立在最小的接口上。每一个接口应该承担一种相对独立的角色，不干不该干的事，该干的事都要干。

●在面向对象编程语言中，实现一个接口就需要实现该接口中定义的所有方法，因此大的总接口使用起来不一定很方便，为了使接口的职责单一，需要将大接口中的方法根据其职责不同分别放在不同的小接口中，以确保每个接口使用起来都较为方便，并都承担某一单一角色。接口应该尽量细化，同时接口中的方法应该尽量少，每个接口中只包含一个客户端（如子模块或业务逻辑类）所需的方法即可，这种机制也称为“定制服务”，即为不同的客户端提供宽窄不同的接口。

设计模式的原则—接口隔离原则(ISP)

- 优势：
  - 减少耦合：系统各部分之间的依赖关系减少
  - 提高灵活性：更容易修改和扩展
  - 降低实现负担：类不需要实现它们不需要的方法
  - 提高可读性：小而专注的接口更易于理解和维护
- 实践：
  - 评估现有接口，识别"胖接口"
  - 根据客户端需求将大接口拆分为更小的接口
  - 使用接口继承来创建更具体的接口
  - 考虑使用适配器模式来处理遗留接口

设计模式的原则—组合聚合复用原则(CARP)



●组合/聚合复用原则 (CARP)： 尽量使用组合(composition)或聚合(aggregation)，而不是继承(inheritance)来实现代码复用。

●在面向对象设计中，可以通过两种方法在不同的环境中复用已有的设计和实现，即通过组合/聚合关系或通过继承，但首先应该考虑使用组合/聚合，组合/聚合可以使系统更加灵活，降低类与类之间的耦合度，一个类的变化对其他类造成的影响相对较少；其次才考虑继承，在使用继承时，需要严格遵循里氏代换原则，有效使用继承会有助于对问题的理解，降低复杂度，而滥用继承反而会增加系统构建和维护的难度以及系统的复杂度，因此需要慎重使用继承复用。

设计模式的原则—组合聚合复用原则(CARP)



●组合/聚合复用原则 (CARP)： 尽量使用组合(composition)或聚合(aggregation)，而不是继承(inheritance)来实现代码复用。

●在面向对象设计中，两种方法在不同的环境中复用已有的设计和实现，组合/聚合关系或继承，首先考虑组合/聚合，组合/聚合更加灵活，降低类与类之间的耦合度，一个类的变化对其他类造成的影响相对较少；其次考虑继承。

●通过继承进行复用的问题在于会破坏系统的封装性，会将基类的实现细节暴露给子类，由于基类的内部细节通常对子类来说是可见的，所以这种复用又称“白箱”复用，如果基类发生改变，那么子类的实现也不得不发生改变；从基类继承而来的实现是静态的，不可能在运行时发生改变，没有足够的灵活性。

●组合或聚合关系可以将已有的对象（也可称为成员对象）纳入到新对象中，使之成为新对象的一部分，因此新对象可以调用已有对象的功能，可以使得成员对象的内部实现细节对于新对象不可见，这种复用又称为“黑箱”复用，相对继承关系，其耦合度相对较低，成员对象的变化对新对象的影响不大，在新对象中根据实际需要有选择性地调用成员对象的操作；合成复用可以在运行时动态进行，新对象可以动态地引用与成员对象类型相同的其他对象。

设计模式的原则—组合聚合复用原则(CARP)

- 优势：
  - 减少耦合：组合/聚合比继承的耦合度更低
  - 更灵活：可以在运行时改变行为
  - 避免继承层次过深：继承层次过深会导致系统难以理解和维护
  - 打破封装：继承会暴露父类实现细节给子类
  - 更易测试：组合的对象可以更容易被模拟或替换
- 实践：
  - 不要完全避免继承 - 当存在真正的"is-a"关系时，继承仍然是合适的
  - 组合可能会增加系统中对象的数量
  - 组合需要更多的委托代码

设计模式的原则—迪米特法则(LoD)



●也称为最少知识原则（Least Knowledge Principle，LKP）

●一个对象应该对其他对象有最少的了解。通俗地讲，一个类应该对自己需要耦合或调用的类知道得最少，你（被耦合或调用的类）的内部是如何复杂都和我没关系，那是你的事情，我就知道你提供的public方法，我就调用这么多，其他的一概不关心。

设计模式的原则—迪米特法则(LoD)

- 优势：
  - 降低耦合度：减少了对象之间的依赖
  - 提高模块独立性：每个模块相对独立，修改一个模块对其他模块影响小
  - 提高可维护性：系统更容易理解和修改
  - 提高可重用性：模块间的依赖减少，重用性提高
- 实践：
  - 适用于：大型复杂系统的设计；需要高度模块化的系统；需要频繁修改和维护的系统
  - 过度应用可能导致：需要编写大量包装方法；系统效率可能略有下降；设计可能变得过于复杂

设计模式的类型

●1. 构造式：对象创建的过程处理

设计模式	简要说明
抽象工厂(Abstract Factory)	提供创建相关的一组对象的接口，不需要指定它们的具体实现。
构造器(Builder)	将对象的结构与它的表示分离，可用同样的结构得到不同的表示。
工厂方法(Factory Method)	定义创建对象的接口，但由子类来决定实例化。
原型(Prototype)	用原型实例指定创建对象的种类，并通过拷贝原型来创建新的对象。
单例(Singleton)	保证一个类仅有一个实例，提供访问它的全局访问点。

设计模式的类型

●2.结构式：处理对象/类的组合

设计模式	简要说明
适配器(Adapter)	把类的接口转换成另外一个接口，解决接口不兼容问题。
桥接(Bridge)	将对象的抽象和实现部分进行分离，使它们能独立变化。
合成(Composite)	把对象组织成树形结构来表示层次关系，使对单个和复合对象的使用具有一致性。
装饰器(Decorator)	动态地为对象添加新的操作功能。
外观(Facade)	把子系统中的各个接口统一为一致的接口。
享元(Flyweight)	运用共享技术支持细粒度的对象集合。
代理(Proxy)	以对象的名义对另一个对象进行访问。

设计模式的类型

●3. 行为式：处理对象间的交互方式和任务分布

设计模式	简要说明
责任链(Chain of Responsibility)	将不同对象对请求的处理形成一个链，使它们都有机会检查请求，从而解除请求和处理者之间的耦合（类似异常的抛出）。
命令(Command Processor)	把请求封装为一个对象，可用不同的请求对客户进行参数化控制。
解释器(Interpreter)	在给定语言和文法的情况下，建立一个该语言的解释器。
迭代器(Iterator)	在不暴露对象内部结构的情况下，循环访问一个对象的集合。
协调器(Mediator)	使用一个中介，在对象不需要显式引用的情况下维持交互关系，从而达到维持对象松散耦合的作用。
备忘录(Memento)	在不破坏封装的前提下，获得对象内部状态的访问和控制权。
观察者(Observer)	建立对象与依赖它的对象间的变化-通知机制（MVC模式）。
状态(state)	使对象在其内部状态发生变化时发生类似类行为的改变。
策略(Strategy/Policy)	定义可以替换的算法，使得算法的变化可以独立于用户。
模板方法(Template Method)	定义某个算法的计算过程框架，具体计算步骤到子类中加以实现，在不改变算法构架的同时改变某些计算步骤的实现。
中介者(Mediator)	用中介对象来封装一系列交互对象，它使对象之间不需要显式地相互引用，构成松散耦合，可独立地改变它们之间的交互。

Singleton 模式

保证一个类仅有一个实例，并提供一个访问它的全局访问点。

- 控制访问唯一实例。因为Singleton 类封装它的唯一实例，所以它可以严格的控制应用的访问。
- Singleton 模式使用静态数据成员和成员函数，得到全局变量应用，可保证唯一存储，避免混乱的对象空间。
- 可以在需要时，用相同的方法改变Singleton类的实例数量。

Singleton 模式

- 查看 Singleton设计模式用例.cpp

Return
uniqueInstance

Singleton
+void* operator new(unsigned int)
+static GetInstance()
-singleton static *s

Singleton 模式

●适用场景:

1. 某个软件组件需要访问数据库。为此，可能希望整个组件只创建一个数据库连接；

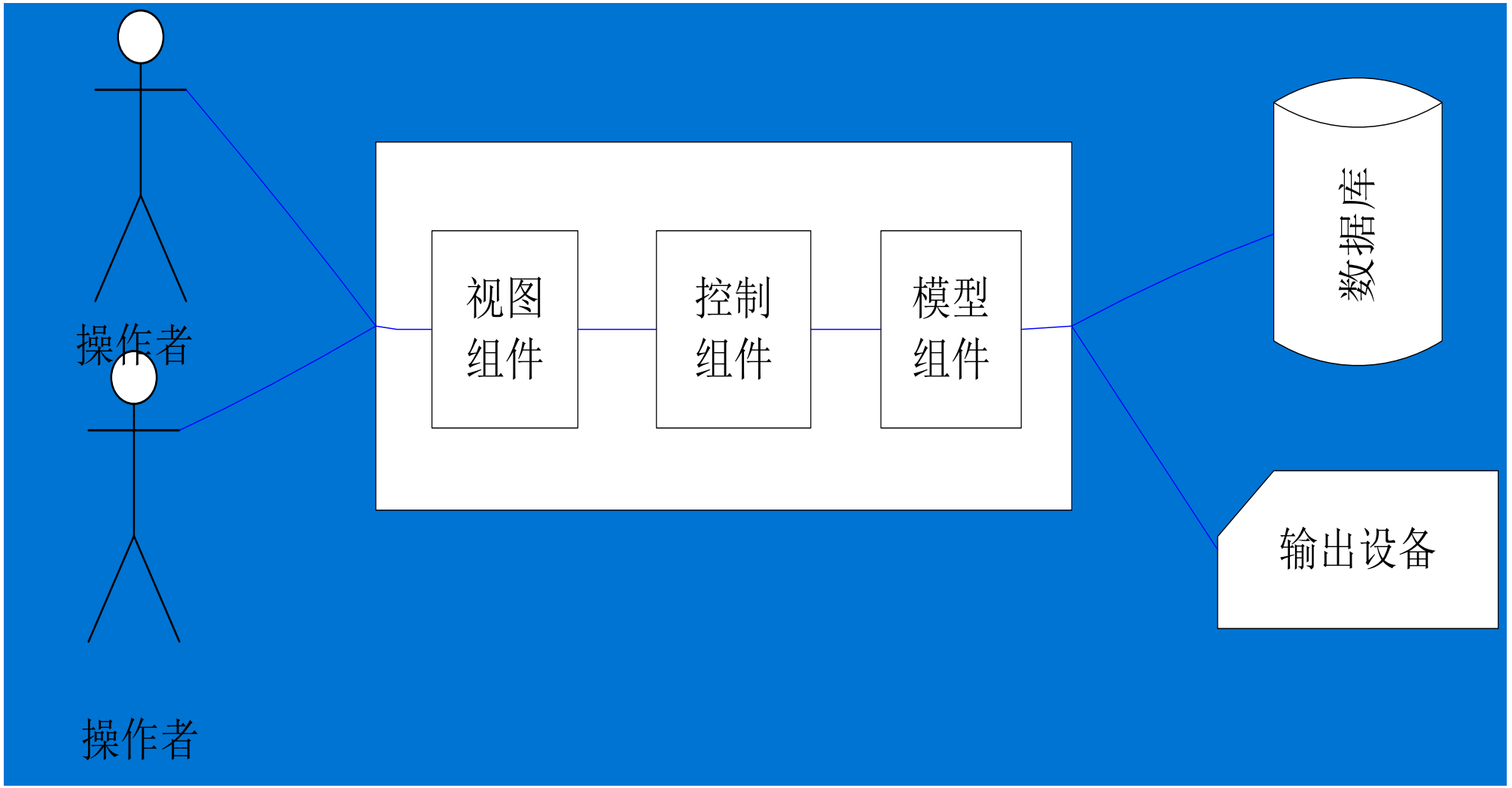
2. 整个系统的主控类，一般只能创建一个。

MVC 模式

●基本概念：MVC（Model-View-Controller,模型-视图-控制器模式）用于表示一种软件架构模式。把软件系统分为三个部分：模型，视图和控制器。

●目的：实现一种动态的程序设计，是后续对程序的修改和扩展简化，并且使程序某一部分的重复利用成为可能。

MVC 模式下的信息管理系统



MVC 模式 View组件

●作用：实现业务数据的输入和显示，外部对象与系统进行交互和通信要通过视图层。一般视图就是为操作者显示的窗口界面，操作者通过该窗口来进行系统管理。

●职责：与操作者进行交互

●功能：

(1)提供数据输入的机制。如表单。

(2)显示业务数据。

1)列表方式　　2)详细方式

MVC 模式 Model组件

●作用：用于封装与应用程序的业务逻辑相关的数据以及对数据的处理方法。模型有对数据直接访问的权利，模型不依赖与视图和控制也就是他不关心如何被显示和被操作。

●职责：业务处理。

●主要功能：

(1)表达业务数据：表达存储在数据库中的业务数据。

(2)业务数据持久化：将业务数据保存到数据库中。

(3) 业务处理方法：管理系统的核心功能是模拟业务代理，代替人工的处理模式，实现信息管理的高效率和低成本。

MVC 模式 Controller组件

●职责：协调View和Mode组件，控制整个系统的流程,接收客户端请求,并调用Model组件对其作出响应。

●主要功能:

(1) 取得View提交的数据

(2) 验证View数据的合法性(形式和业务)

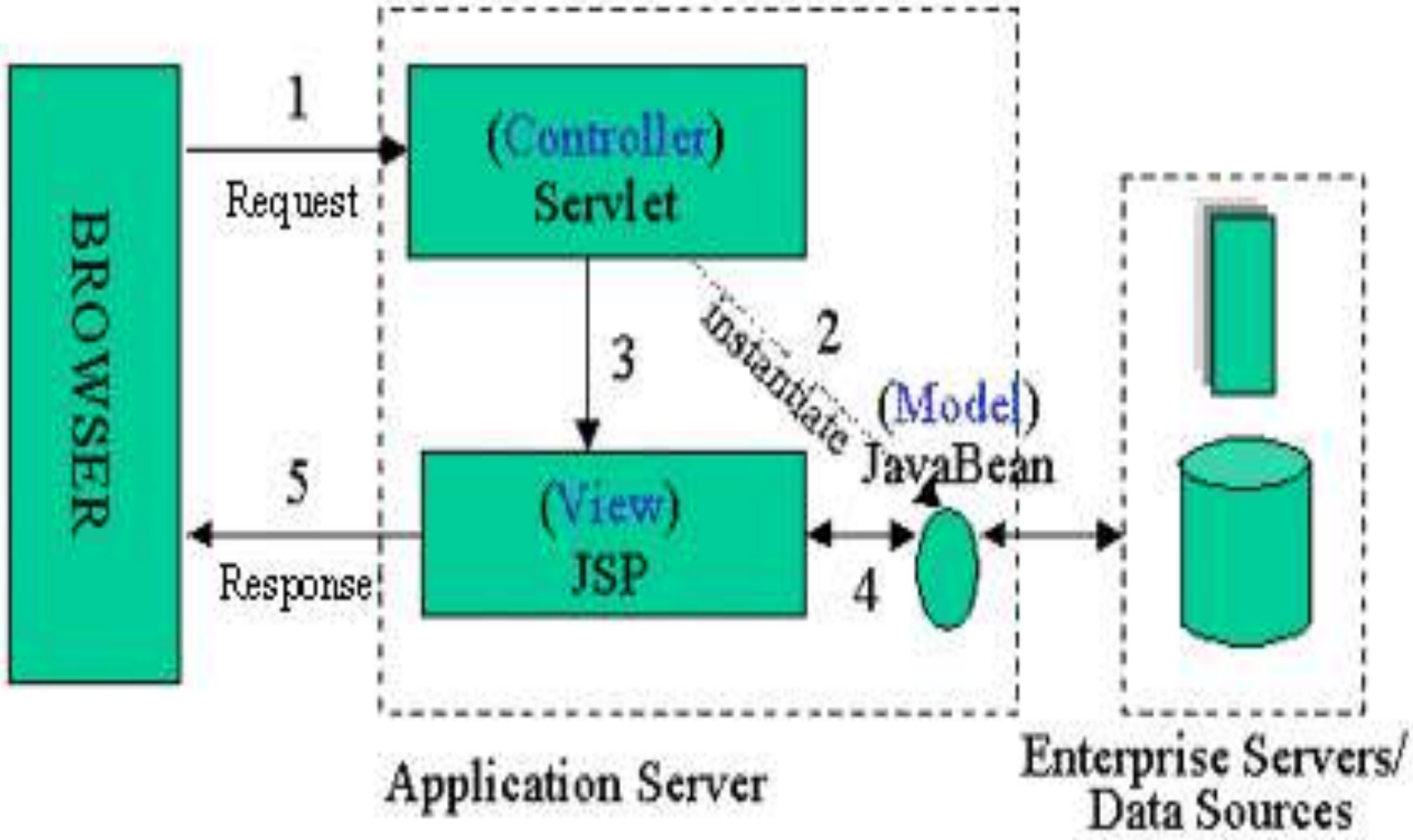
(3) View数据的类型转换

(4) 调用Model业务处理方法

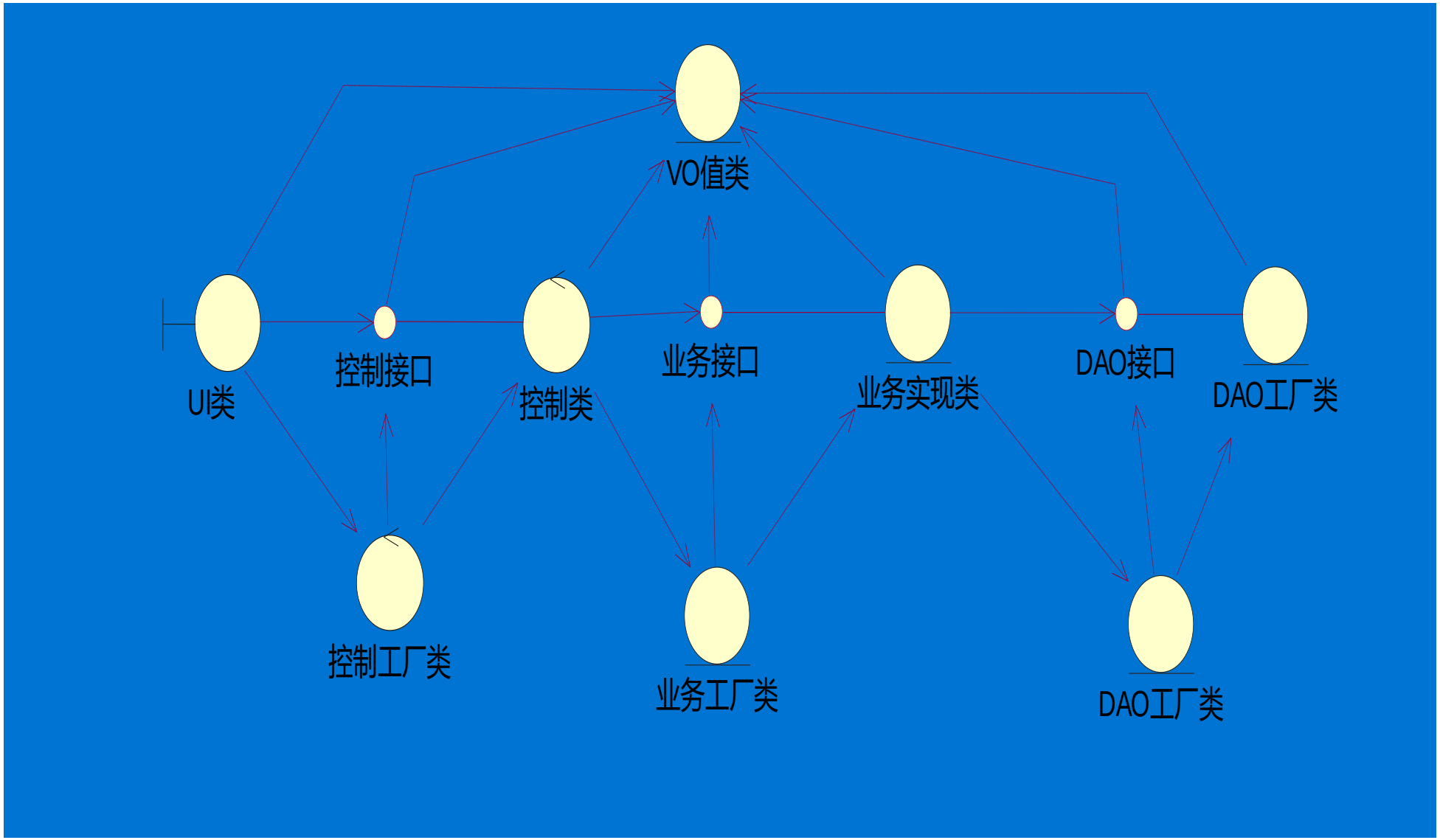
(5) 保存Model业务数据给View组件

(6) 导航到指定View组件。

MVC 模式 JavaEE下的MVC



MVC 模式 详细的MVC分层架构



工厂模式

●工厂模式是一种创建型设计模式，它提供了一种创建对象的最佳方式，而无需向客户端暴露创建逻辑。工厂模式通过定义一个创建对象的接口，让子类决定实例化哪一个类。

●1 简单工厂模式

●2 工厂方法模式

●3 抽象工厂模式

工厂模式-简单工厂

●简单工厂模式又称为静态工厂方法模式，它定义一个工厂类，根据传入的参数不同返回不同的实例。

●工厂类（Factory）：负责创建具体产品的静态方法

●抽象产品（Product）：定义产品的接口

●具体产品（ConcreteProduct）：实现抽象产品接口的具体类

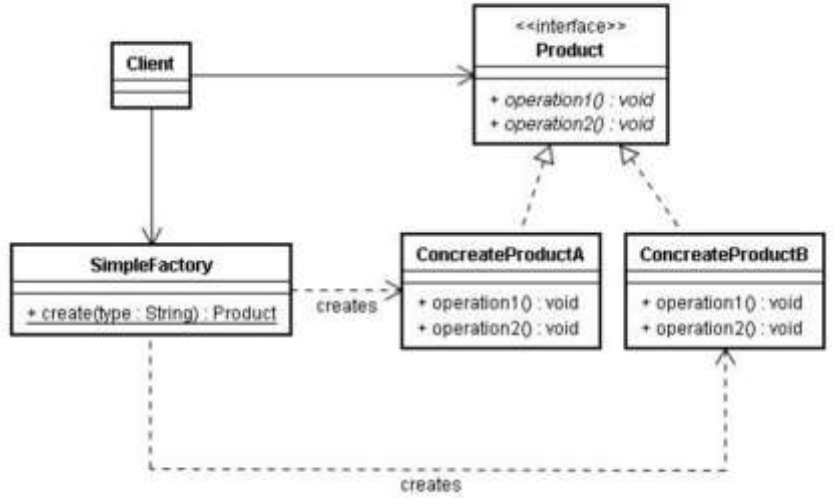
●优点：

●客户端与具体产品解耦；客户端不需要知道具体产品的类名

●缺点：

●工厂类集中了所有产品的创建逻辑，职责过重

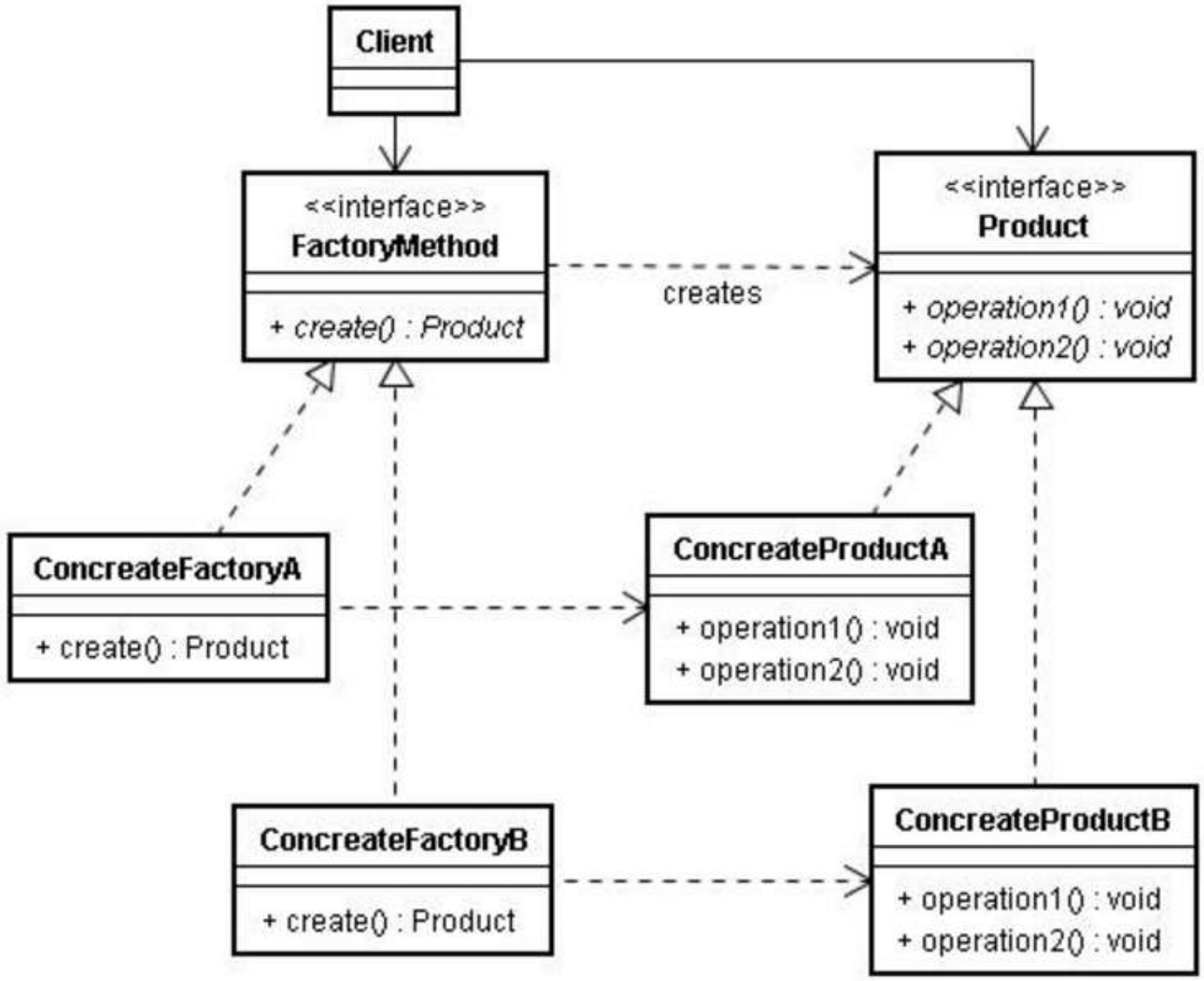
●添加新产品需要修改工厂类，违反开闭原则



工厂模式-工厂方法

- 定义一个用于创建对象的接口，让子类决定实例化哪一个类。工厂方法使一个类的实例化延迟到其子类。
  - Product（抽象产品）：定义产品的接口
  - ConcreteProduct（具体产品）：实现抽象产品接口
  - Factory（抽象工厂）：声明工厂方法
  - ConcreteFactory（具体工厂）：实现工厂方法，返回具体产品实例
- 优点：完全符合开闭原则，新增产品只需添加新的工厂类；符合单一职责原则；客户端只需知道抽象工厂和抽象产品
- 缺点：每增加一个产品就要增加一个具体工厂类，增加了系统复杂度；引入了抽象层，增加了理解难度
- 适用场景：客户端不知道它所需要的对象的类；类将创建对象的任务委托给多个工厂子类中的某一个

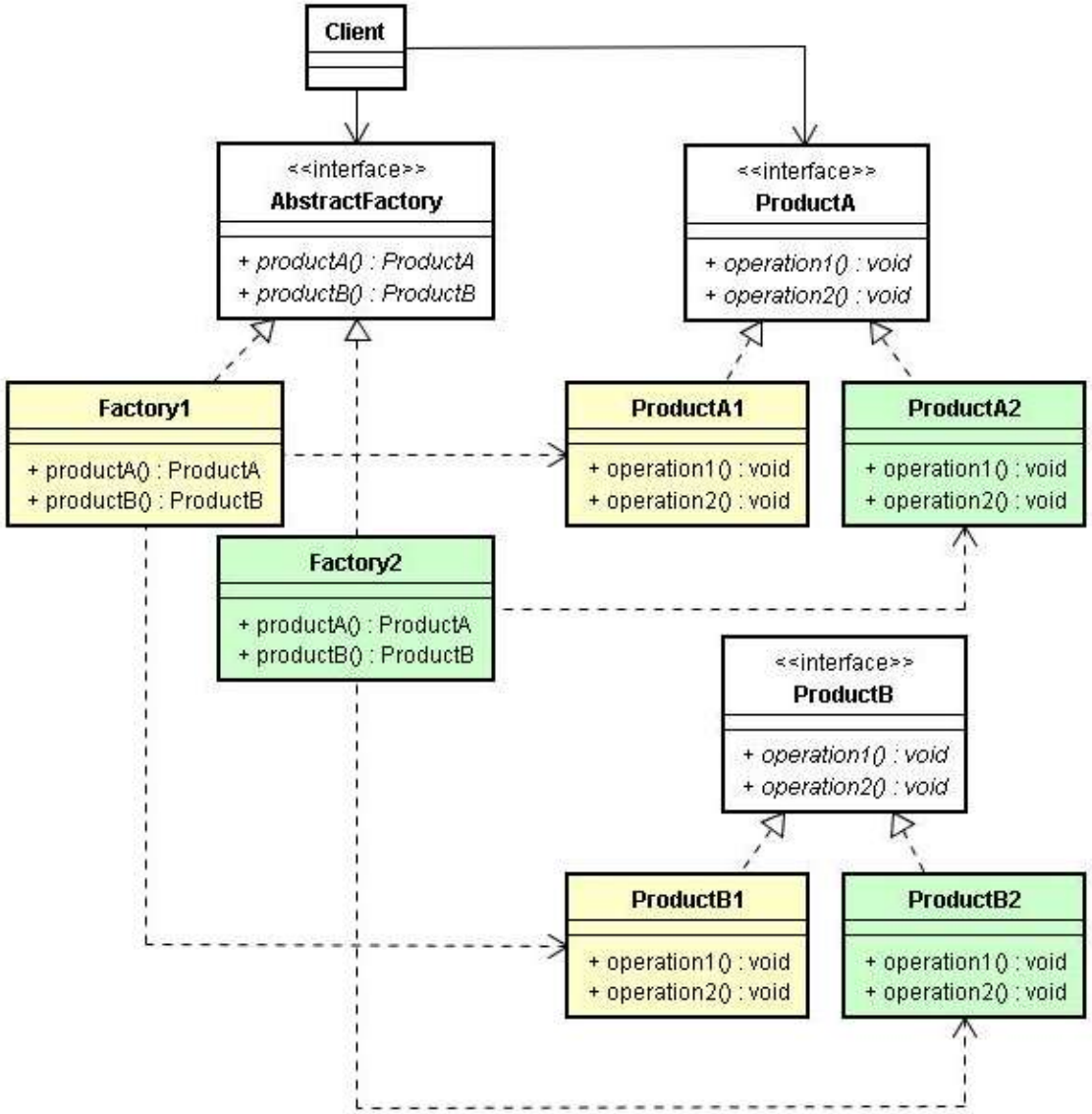
工厂模式-工厂方法


Client
<<interface>>
FactoryMethod
+ create() : Product
creates
<<interface>>
Product
+ operation1() : void
+ operation2() : void
ConcreateFactoryA
+ create() : Product
ConcreateProductA
+ operation1() : void
+ operation2() : void
ConcreateFactoryB
+ create() : Product
ConcreateProductB
+ operation1() : void
+ operation2() : void

工厂模式-抽象工厂

- 提供一个创建一系列相关或相互依赖对象的接口，而无需指定它们具体的类。
  - AbstractFactory（抽象工厂）：声明创建抽象产品的方法
  - ConcreteFactory（具体工厂）：实现创建具体产品的方法
  - AbstractProduct（抽象产品）：为每种产品声明接口
  - ConcreteProduct（具体产品）：定义具体工厂创建的具体产品对象
- 优点：隔离了具体类的生成；易于交换产品系列；保证客户端始终只使用同一个产品族中的对象
- 缺点：难以支持新种类的产品；系统扩展困难，增加新产品需要修改抽象工厂接口
- 适用场景：系统需要独立于产品的创建、组合和表示；系统需要配置多个产品族中的一个来使用；需要提供一个产品类库，且只想显示它们的接口而非实现

工厂模式-抽象工厂


Client
<<interface>>
AbstractFactory
+ productA() : ProductA
+ productB() : ProductB
<<interface>>
ProductA
+ operation1() : void
+ operation2() : void
Factory1
+ productA() : ProductA
+ productB() : ProductB
ProductA1
+ operation1() : void
+ operation2() : void
ProductA2
+ operation1() : void
+ operation2() : void
Factory2
+ productA() : ProductA
+ productB() : ProductB
<<interface>>
ProductB
+ operation1() : void
+ operation2() : void
ProductB1
+ operation1() : void
+ operation2() : void
ProductB2
+ operation1() : void
+ operation2() : void

工厂模式-小结



- 工厂模式中，重要的是工厂类，而不是产品类。产品类可以是多种形式，多层继承或者是单个类都是可以的。但要明确，工厂模式的接口只会返回一种类型的实例，这是在设计产品类的时候需要注意的。
- 使用工厂模式，返回的实例一定是工厂创建的，而不是从其他对象中获取的

工厂模式-小结

- 简单工厂： 用来生产同一等级结构中的任意产品。
  （对于增加新的产品，无能为力）
- 工厂方法：用来生产同一等级结构中的固定产品。
  （支持增加任意产品）
- 抽象工厂：用来生产不同产品族的全部产品。
  （对于增加新的产品，无能为力；支持增加产品族）

谢 谢!



